

BLE TAP EXCHANGE

Firmware Architecture & Engineering Reference

ESP32-C3 · ESP-IDF 5.4.3 · FreeRTOS · NimBLE

Field	Detail
Revision	1.0 — May 2026
MCU	ESP32-C3 Super Mini (RISC-V, 160 MHz, 400 KB SRAM, 4 MB Flash)
Framework	ESP-IDF 5.4.3
RTOS	FreeRTOS (bundled with ESP-IDF)
BLE Stack	NimBLE (not Bluedroid)
Hardware	SSD1306 0.96" OLED (I2C), 4× tactile buttons, passive piezo buzzer
Language	C — no C++, no Arduino framework
License	MIT
Current state	All 9 subsystem tests passing. Full UI integration in progress.

NOTE: *This document explains every architectural and protocol decision made in this firmware — not just what the code does, but why each decision was made. Read it before reading the source code.*

1. What This Project Is

BLE Tap Exchange is firmware for a wrist-worn device that lets two people exchange contact information — name, phone number, job title — by simultaneously pressing a button on each device. The exchange happens over Bluetooth Low Energy. No phone app is required. No pairing is required. No server is involved.

The core engineering problem is this: two independent embedded devices, with no prior knowledge of each other, must detect that they were activated at approximately the same moment, decide which one initiates the connection, exchange structured data reliably, and store it to non-volatile memory — all without user configuration, all in under ten seconds, and all without ever connecting to a device that was not part of the same tap event.

The current prototype uses a button press as the tap trigger. The v2 firmware replaces this with a LIS3DH accelerometer interrupt — when two wrists make contact, both IMUs register the collision at the same instant, and that shared timestamp becomes the basis of the connection security. The button-to-IMU swap is a single driver replacement; nothing above the driver layer changes.

This document covers the firmware architecture, the BLE protocol design, every key engineering decision with its rationale, and the nine-stage hardware test procedure used to verify the system incrementally.

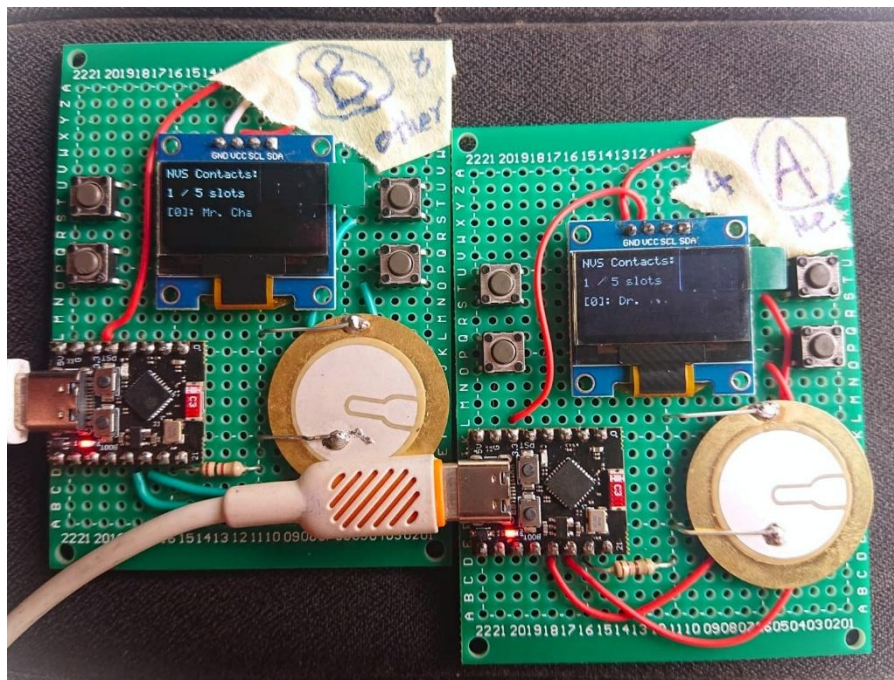


Figure 1: test setup

2. Firmware Architecture

2.1 The four-layer model

Every file in this project belongs to exactly one of four layers. Each layer may only call the layer directly below it. No layer skips a level. This constraint is enforced by code review, not by the compiler, but it is the single most important rule in the codebase.

Layer	Folder	Responsibility	Knows about
4 — Application	app/	User-visible screens and navigation. What the user sees and does.	Nothing about hardware. Calls services only.
3 — Services	services/	Business logic. What a contact exchange IS. What an alarm DOES.	Nothing about GPIO, I2C, or BLE internals. Calls drivers only.
2 — Drivers	drivers/	How to operate one specific piece of hardware or radio stack.	Only its own peripheral. No application logic.
1 — Hardware	(physical)	SSD1306, buttons, buzzer, BLE radio, RTC.	Not firmware — this is silicon.

DECISION: This layering was chosen so that individual components can be swapped without cascading changes. The v2 hardware upgrade replaces the SSD1306 with a round GC9A01 colour display — only `ssd1306_driver.c` is replaced. Nothing above the driver layer notices. The same principle applies to the RTC: the internal ESP32-C3 RTC can be replaced with a DS3231 external RTC by replacing `rtc_driver.c` only. All callers of `rtc_get_time()` and `rtc_set_hms()` are unaffected.

NOTE: RULE: If you find yourself calling a driver function directly from a service or app file, stop. You are breaking the layer contract. Add a service function in between.

2.2 Project structure

```

ble-tap-exchange/
├── main/
│   └── main.c                ← entry point. creates tasks. nothing else.
├── components/
│   ├── common/
│   │   └── include/
│   │       ├── app_config.h  ← ALL constants. pins, stack sizes, timeouts.
│   │       ├── app_events.h  ← ALL event/message types for queues.
│   │       └── wifi_sync.h/.c ← one-shot NTP sync at boot (prototype only)
│   ├── drivers/
│   │   ├── ssd1306/          ← DONE & TESTED (test 01)
│   │   ├── button/          ← DONE & TESTED (test 02)
│   │   ├── buzzer/          ← DONE & TESTED (test 03)
│   │   ├── rtc/             ← DONE & TESTED (test 04)
│   │   └── ble/              ← DONE & TESTED (tests 06-09)
│   ├── services/
│   │   ├── ble_service/      ← DONE & TESTED (test 09)
│   │   ├── time_service/     ← planned v1 UI
│   │   ├── alarm_service/    ← out of scope v1
│   │   ├── stopwatch_service/ ← out of scope v1
│   │   └── ui_service/       ← planned v1 UI
│   └── tasks/
│       └── tasks.c            ← all 5 FreeRTOS task functions
└── tests/
    ├── 01_display/ ... 09_ble_tapshare/
    └── test_config.h.template
  
```

2.3 FreeRTOS task model

Each major responsibility runs as an independent FreeRTOS task. Tasks never call each other directly. All inter-task communication flows through queues or event groups. This is not optional style — it is the only safe way to share data between concurrent execution contexts without data races.

Task	Priority	Stack	Responsibility
ble_task	5 (highest)	6144 B	Drives NimBLE host, receives BLE events, runs ble_service state machine
input_task	4	2048 B	Reads button GPIOs, debounces, detects long-press, routes events to queues
ui_task	3	4096 B	Redraws OLED on second ticks and button events
alarm_task	3	2048 B	Waits on alarm event group bit, drives buzzer
time_task	2	2048 B	Increments RTC cache every second, sets event group bits

DECISION: BLE is given the highest priority because NimBLE has internal timing requirements. If ble_task is starved, the host controller interface (HCI) can fall behind, causing connection drops or assert failures. Input is second because users notice button lag immediately. Display is lower — a frame arriving 50 ms late is invisible.

The golden FreeRTOS pattern used throughout

Every task in this firmware follows the same structure. Memorise this:

```
static void my_task(void *pv) {
    my_service_init();           // one-time setup
    while (1) {
        // BLOCK here — zero CPU used while waiting
        SomeType_t item;
        if (xQueueReceive(g_queue, &item, portMAX_DELAY) == pdPASS) {
            process(item);        // do the work
        }
    }
    // never returns
}
```

WARNING: Never write a while(1) without a blocking call or vTaskDelay inside it. A task that never yields starves every task at lower or equal priority. The hardware watchdog will eventually reset the chip.

2.4 Inter-task communication

Two FreeRTOS primitives are used for all inter-task communication:

Primitive	Handle	Item type	Who sends	Who receives
Queue	g_input_queue	button_event_t	input_task (from ISR via xQueueSendFromISR)	ui_task, ble_service
Queue	g_ble_event_queue	ble_event_t	ble_driver callbacks	ble_task → ble_service
Event group	g_time_event_grp	TIME_EVT_* bits	time_task	ui_task, alarm_task

DECISION: Queues carry data (button press events, BLE exchange results). Event groups carry signals (the second just ticked, the alarm just fired). The distinction matters: an event group bit is stateless after it is read. A queue item contains a payload struct and can be inspected. Using a queue for time ticks would waste memory. Using an event group for BLE exchange results would lose the contact data payload.

2.5 Shared types — app_config.h and app_events.h

Every constant in the system lives in app_config.h. Every event type and message struct lives in app_events.h. These two files are the vocabulary every other file speaks. They are created before any driver or service code.

NOTE: When a pin assignment changes for the v2 PCB, one line changes in app_config.h. Nothing else touches it. This is the point of centralising constants — hardware changes stay in one file.

Key constants defined in app_config.h:

Constant	Value	Purpose
PIN_BTN_SELECT	9	Boot-safe GPIO. Used for tap-to-share trigger and UI navigation.
TAPSHARE_RSSI_THRESHOLD	-65 dBm	Maximum range gate. Rejects peers further than ~30 cm.
TAPSHARE_TIME_WINDOW_S	5 s	Tap synchrony gate. Rejects taps more than 5 s apart.
TAPSHARE_MAX_CONTACTS	5	NVS contact slots. FIFO rotation when full.
BLE_ADV_INTERVAL_MS	100 ms	Advertising interval. ~200 ms average discovery latency.
TAPSHARE_ADV_MAGIC	0xC0	First byte of manufacturer data. Identifies our advertisement.
STACK_BLE	6144 B	Measured via watermark. NimBLE needs at least 4 KB; 6 KB gives headroom.
LONG_PRESS_MS	800 ms	Hold duration threshold for long-press detection in input_task.

3. Display Driver — SSD1306

3.1 Design decisions

DECISION: Custom framebuffer driver instead of LVGL. LVGL is a capable UI framework but consumes 80-150 KB of RAM. NimBLE already uses 85-100 KB. The ESP32-C3 has 400 KB total SRAM. Running both would leave less than 150 KB for all tasks, stacks, queues, and NVS buffers — unsafe margins. A custom driver was written instead.

The driver maintains a 1024-byte framebuffer (128 × 64 pixels, 1 bit per pixel, 8 pixels per byte in vertical pages). All drawing operations write to this buffer in RAM. Nothing is sent to the display until `ssd1306_flush()` is called. This batches all I2C traffic into a single transaction per frame.

Feature	Implementation
Framebuffer	<code>uint8_t framebuf[8][128]</code> — 8 pages × 128 columns × 8 vertical pixels per page
Dirty tracking	8-bit bitmask per page. <code>flush()</code> skips pages unchanged since last flush.
Primitives	<code>draw_pixel</code> , <code>draw_rect</code> (fill/outline), <code>draw_text</code> , <code>draw_line</code>
Fonts	<code>font_5x7.h</code> — full ASCII. <code>font_digits_large.h</code> — 12×16 segment digits for clock face.
Flush	Single I2C burst per dirty page. Uses ESP-IDF 5.x <code>i2c_master_transmit()</code> .
Opaque handle	<code>ssd1306_handle_t</code> hides internal struct. Callers cannot access framebuffer directly.

DECISION: The opaque handle pattern (`typedef struct ssd1306_dev_t* ssd1306_handle_t`) was chosen so that in v2, when the SSD1306 is replaced with a GC9A01 round colour display, the service layer above sees no API change. It still calls `ssd1306_flush()`. Only the driver implementation changes.

NOTE: Draw order matters. Drawing a white rectangle after drawing white text will paint over the text silently — both operations write to the same framebuffer bits. This was discovered during test 01 (Test 12 — white text under white rectangle). A dot-grid side-by-side comparison was used to diagnose it.

3.2 ESP-IDF 5.x I2C API

ESP-IDF 5.x replaced the legacy `i2c_driver_install()` API with a new master bus model. Any code using the old API will not compile on 5.4.3.

Old API (4.x — do not use)	New API (5.x — use this)
<code>i2c_driver_install()</code>	<code>i2c_new_master_bus()</code>
<code>i2c_cmd_handle_t</code> pattern	<code>i2c_master_transmit()</code> / <code>i2c_master_receive()</code>
<code>i2c_master_cmd_begin()</code>	Removed — single-call model replaces it

4. Button Driver

4.1 ISR-to-queue pattern

Four GPIO inputs with internal pull-ups. The fundamental challenge is debounce: a mechanical button makes and breaks contact 5-20 times in the first 5-20 ms of a press. Without handling this, one physical press registers as 10 software events.

DECISION: *Debounce is handled in the task layer, not the ISR. The ISR sends only the GPIO number and a timestamp to a queue and returns immediately. `input_task` receives this, waits `BTN_DEBOUNCE_MS` (50 ms), then checks if the pin is still in the pressed state. If yes, it is a real press. If no, it discards the event. This keeps the ISR tiny and the debounce logic testable.*

```
// ISR - runs in interrupt context. Must be in IRAM.
static void IRAM_ATTR btn_isr_handler(void *arg) {
    button_id_t id = (button_id_t)(uintptr_t)arg;
    BaseType_t hp = pdFALSE;
    xQueueSendFromISR(s_output_queue, &id, &hp);
    portYIELD_FROM_ISR(hp);    // yield if a higher-priority task woke
}
```

WARNING: *ISR functions run on a tiny dedicated stack with interrupts partially disabled. They cannot call `ESP_LOG`, `malloc()`, `vTaskDelay()`, or any standard FreeRTOS function. Only use `xQueueSendFromISR()`, `xSemaphoreGiveFromISR()`, and similar `_FromISR` variants.*

4.2 Long-press detection

Long-press is detected in `input_task`, not in the ISR. On receiving a `BTN_EVT_PRESS`, `input_task` waits up to `LONG_PRESS_MS` for a `BTN_EVT_RELEASE` on the same queue. If the release does not arrive within the timeout, the event type is promoted to `BTN_EVT_LONG_PRESS` before being forwarded to the UI layer.

5. Buzzer Driver

A passive piezo buzzer driven by the ESP32-C3 LEDC (LED PWM Controller) peripheral. LEDC generates precise PWM frequencies entirely in hardware — the CPU does nothing after configuration.

DECISION: LEDC was chosen over a software timer approach because LEDC produces a clean square wave at any frequency without jitter. A software timer fires at FreeRTOS tick resolution (1 ms at 1 kHz tick rate), which introduces audible artifacts at higher frequencies. LEDC is also zero CPU overhead once configured.

Sound	Frequency / Pattern	Trigger
Button feedback	1000 Hz, 50 ms	Every short button press
Mode switch	1200 Hz, 50 ms	MODE button press
Long-press detected	800 Hz rising, 200 ms	SELECT long-press — sharing activated
Scan pulse	600 Hz, 80 ms, every 2 s	While in SHARING state (searching)
Share success	880 Hz → 1320 Hz, 300 ms each	GATT exchange completed — contact saved
Share failed	Low-high-low, 300 ms	Scan timeout or GATT failure
Cancel sharing	500 Hz, 150 ms	SELECT long-press while in SHARING state

6. RTC Driver and Time Sync

6.1 Internal RTC — intentional imprecision

The ESP32-C3 internal RTC is used for timekeeping. It drifts approximately ± 250 ppm — roughly ± 20 seconds per day. This is intentional for prototype v1.

DECISION: An external DS3231 RTC module (± 2 ppm, ± 0.17 s/day) was evaluated and deferred to v2. The internal RTC is sufficient to demonstrate the watch UI and validate the firmware architecture. Adding an external I2C device adds hardware complexity and cost to a prototype where timing precision is not the engineering question being validated.

6.2 Time-source agnostic API

The RTC driver API was designed to be source-agnostic. The three public functions are `rtc_driver_init()`, `rtc_get_time()`, and `rtc_set_hms()`. Nothing in the service layer above knows whether time comes from the internal RTC, a DS3231, or a GPS module. Replacing the time source means replacing `rtc_driver.c` only.

6.3 WiFi NTP sync — prototype only

On first boot, `wifi_sync.c` connects to a configured WiFi network, performs a single SNTP query, sets the RTC, and disconnects. WiFi is not used again.

DECISION: NTP sync was included for prototype demos where an accurate clock display is useful. In production, time sync would come from a companion phone app over BLE. The NTP path uses `esp_sntp` (built into ESP-IDF) and is isolated in `wifi_sync.c`. The RTC driver API does not know or care how `rtc_set_hms()` was called — making the BLE sync path a drop-in replacement.

NOTE: WiFi credentials are stored in `tests/test_config.h` which is gitignored. The firmware falls back to a manual time display if the file is absent or WiFi is unavailable.

7. BLE Driver — Protocol Design

The BLE driver is the most complex component in the firmware. It operates a radio, implements a proximity detection protocol, manages asynchronous GATT operations across multiple callback contexts, and must never block the other tasks. This section explains every protocol decision in detail.

7.1 Why NimBLE and not Bluedroid

Criterion	NimBLE	Bluedroid
RAM usage	~85-100 KB	~150-200 KB
ESP-IDF 5.x support	First-class, Espressif-maintained	Deprecated for ESP32-C3
Multi-role (Central + Peripheral simultaneously)	Supported	Partial support, unreliable
Code complexity	Callback-driven, predictable	Event-driven, more opaque
RISC-V (ESP32-C3) support	Full	Limited

DECISION: *NimBLE was the only viable choice for this hardware. Bluedroid is deprecated on ESP32-C3 in ESP-IDF 5.x and does not reliably support the simultaneous Central + Peripheral role required by the tap exchange protocol. NimBLE's lower RAM footprint was the deciding factor given the shared memory budget with display framebuffer and task stacks.*

7.2 Dual-role simultaneous operation

Both watches run as Peripheral (advertising) and Central (scanning) simultaneously from the moment sharing is activated. This is the NimBLE multi-role capability.

DECISION: *Dual-role is necessary because at the moment the user presses the button, neither watch knows whether it will be the one to initiate the connection. If only one role were active, one watch would need to advertise and the other scan — requiring the users to designate roles beforehand, which defeats the purpose of a symmetric tap interaction. Both watches advertise and scan simultaneously until the role decision logic (section 7.4) determines which one connects.*

WARNING: *Once a GAP connection is established, scanning AND advertising must stop immediately. The ESP32-C3 NimBLE stack cannot reliably maintain simultaneous scan + advertise + connected state. `ble_driver.c` handles this automatically inside the scan callback before calling `ble_gap_connect()`.*

7.3 Using the advertising packet for collision detection

This is the most important protocol decision in the firmware. The problem it solves: how do two independent watches, with no prior communication, know they were activated at approximately the same moment — and not by someone else nearby who happened to press their button 30 seconds later?

The solution: the tap timestamp is embedded directly in the BLE advertising packet in the manufacturer-specific data field, before any connection is made.

```
// Advertising packet manufacturer data layout:  
// Byte 0: TAPSHARE_ADV_MAGIC (0xC0) — identifies Cylonix advertisement  
// Bytes 1-4: tap_timestamp (uint32_t, little-endian) — esp_timer epoch at  
button press
```

Every watch that is scanning reads this timestamp from the advertisement of every peer it discovers — no connection required. The timestamp is used as gate 3 in the 4-gate filter (section 7.4).

DECISION: *Encoding the tap timestamp in the advertising packet means the timestamp comparison happens before any connection is attempted. A connection that would fail gate 3 is never initiated. This avoids wasting a BLE connection slot and the 200-500 ms connection setup latency on a peer that is not part of the same tap event. It also means the role decision (which watch connects) is made from data already available in RAM — not from a network round trip.*

NOTE: *The manufacturer data magic byte (0xC0) ensures that other devices using manufacturer data in their advertisements do not trigger spurious matches. Gate 1 (service UUID) provides a second layer of filtering before the magic byte is even checked.*

7.4 The 4-gate scan filter

All four gates are evaluated inside the scan callback, before `ble_gap_connect()` is ever called. A peer that fails any gate is silently discarded. No state is updated. No connection is attempted.

Gate	Check	Threshold	Fail action	Rationale
1 — UUID	Advertisement contains TapShare service UUID	Must match exactly	Silent discard	Ignores all non-TapShare BLE devices in range
2 — RSSI	Received signal strength	≥ -65 dBm (~30 cm)	Log and discard	Prevents exchange with a watch across the room
3 — Time window	$ \text{peer_ts} - \text{my_ts} $	≤ 5 seconds	Log and discard	Prevents exchange with a watch activated at a different time
4 — Role	Timestamp comparison for initiator decision	Lower ts = Central	If Peripheral: discard (wait for peer)	Deterministic role assignment without negotiation

DECISION: The RSSI threshold of -65 dBm was chosen empirically. At -65 dBm on the ESP32-C3 with its integrated antenna, the devices are typically 20-40 cm apart — consistent with a wrist-tap gesture. At -70 dBm, devices across a small room begin to pass the gate. The 5-second time window was chosen to accommodate the human reaction time difference between two people trying to press simultaneously — in practice the difference is under 1 second, but 5 seconds provides comfortable margin without allowing false matches.

7.5 Deterministic role assignment

After all 4 gates pass, exactly one watch must initiate the connection. Both watches must independently arrive at the same decision without communicating.

```
if (my_tap_ts < peer_tap_ts) {
    role = CENTRAL;           // I pressed first - I connect
} else if (my_tap_ts > peer_tap_ts) {
    role = PERIPHERAL;       // Peer pressed first - I wait
} else {
    // Identical timestamps (same millisecond) - use MAC address
    // Both watches know both MAC addresses from the advertisement
    role = (memcmp(own_addr, peer_addr, 6) < 0) ? CENTRAL : PERIPHERAL;
}
```

DECISION: *MAC address tiebreaker was chosen for the equal-timestamp case because both watches already have both MAC addresses in memory (own address from NimBLE init, peer address from the scan result). The comparison is a 6-byte memcmp — nanoseconds of execution time, no network round trip, no protocol overhead. Both watches perform the identical memcmp on the same two addresses and reach the same conclusion independently. This is a fully deterministic, zero-communication role negotiation.*

7.6 GATT table design — 3 separate characteristics

The TapShare service exposes three GATT characteristics: Name (32 bytes), Phone (16 bytes), and Title (64 bytes). Each characteristic has both READ and WRITE properties.

Characteristic	UUID	Properties	Max length	Maps to
Name	TAPSHARE_CHR_NAME_UUID	READ WRITE	32 bytes	contact_entry_t.name
Phone	TAPSHARE_CHR_PHONE_UUID	READ WRITE	16 bytes	contact_entry_t.phone
Title	TAPSHARE_CHR_TITLE_UUID	READ WRITE	64 bytes	contact_entry_t.title

DECISION: Three separate characteristics instead of one JSON blob or one concatenated string. The reason is partial failure handling. If the Central reads name and phone successfully but the title read fails (e.g. connection drops), the firmware knows exactly which field failed. The read-first contract (all reads before any write) ensures that if any read fails, neither watch saves incomplete data. A single-blob approach would require parsing a partial blob on failure — fragile and unnecessary.

WARNING: Replace the placeholder UUID byte arrays in `ble_driver.c` with real UUID v4 values from <https://www.uuidgenerator.net/> before testing with other BLE devices. Generic or sequential UUIDs will conflict with other BLE devices in the vicinity.

7.7 GATT exchange sequence — read-first contract

The exchange is fully asynchronous. Each GATT operation fires a callback that queues the next operation. The Central executes this sequence:

```
Step 1: READ peer Name    → gatt_read_cb → advance stage
Step 2: READ peer Phone  → gatt_read_cb → advance stage
Step 3: READ peer Title  → gatt_read_cb → all reads done → begin writes
Step 4: WRITE own Name   → gatt_write_cb → advance stage
Step 5: WRITE own Phone  → gatt_write_cb → advance stage
Step 6: WRITE own Title  → gatt_write_cb → EXCHANGE COMPLETE
        → send BLE_EVT_EXCHANGE_COMPLETE with full peer profile to queue
```

DECISION: Read-first contract: all three reads must succeed before any write is attempted. If any read fails, `BLE_EVT_EXCHANGE_FAILED` is sent and the chain stops — no writes are attempted. This ensures that either both watches complete the full exchange or neither saves anything. There are no asymmetric half-exchanges where one watch has the other's contact but not vice versa.

7.8 NimBLE initialisation sequence

The NimBLE init sequence is fixed. These calls must happen in this exact order:

1. `nvs_flash_init()` — NimBLE requires NVS. Must be called before any BLE init.
2. `esp_nimble_hci_and_controller_init()` — initialises the HCI transport and BLE controller.
3. `nimble_port_init()` — initialises the NimBLE host stack.
4. `ble_svc_gap_init()` + `ble_svc_gatt_init()` — required NimBLE service helpers.
5. `ble_svc_gap_device_name_set()` — sets the device name visible in scans.
6. `ble_gatts_count_cfg()` + `ble_gatts_add_svcs()` — registers our 3-characteristic GATT service.
7. `ble_hs_cfg.sync_cb = on_sync_cb` — registered callback, fires when stack is ready.
8. `nimble_port_freertos_init(ble_host_task)` — creates the NimBLE internal FreeRTOS task.

8. BLE Service — State Machine

ble_service.c sits at the service layer. It owns the TapShare state machine, NVS contact storage, and own-profile management. It never calls NimBLE APIs directly — all BLE operations flow through ble_driver.c.

8.1 State machine

State	Display	Description
BLE_STATE_IDLE	Watch face	BLE radio off. Default state after boot.
BLE_STATE_SHARING	Watch face + icon	Advertising + scanning. Buzzer scan pulse every 2 s.
BLE_STATE_CONNECTING	Watch face	Peer found, all 4 gates passed. GAP connect in progress.
BLE_STATE_EXCHANGING	Watch face	Connected. Reading + writing GATT characteristics.
BLE_STATE_DONE	Share result	Exchange complete. 'SAVED: [name]' shown for 4 s.
BLE_STATE_FAILED	Error screen	Exchange failed or scan timeout. Auto-returns to IDLE.

8.2 State transition table

From	Trigger	To	Action
IDLE	SELECT long-press	SHARING	ble_driver_start_share_mode(tap_ts). Start scan timeout timer.
SHARING	SELECT long-press (cancel)	IDLE	ble_driver_stop_share_mode(). Cancel timer. Cancel buzzer.
SHARING	Peer found (4 gates pass)	CONNECTING	ble_gap_connect(). Stop adv + scan.
SHARING	Scan timeout (8 s)	FAILED	Error beep. Auto-idle after display timeout.
CONNECTING	GAP connect success	EXCHANGING	ble_driver_start_gatt_exchange(conn_handle, own_profile).
CONNECTING	GAP connect failed	FAILED	Log. Auto-idle.
EXCHANGING	EXCHANGE_COMPLETE event	DONE	save_contact_to_nvs(). Show 'SAVED:[name]'. Disconnect.
EXCHANGING	EXCHANGE_FAILED event	FAILED	Disconnect. Error beep. Auto-idle.
DONE	4 s timer expires	IDLE	Ready for next exchange immediately.
FAILED	Display timeout	IDLE	Auto-recovery. Ready for retry.

8.3 NVS contact storage — power-loss safe write sequence

Contacts are stored in NVS under the 'tapexchange' namespace. A 5-slot FIFO circular buffer is used. When all slots are full, the oldest contact is overwritten.

The write sequence for each contact:

9. `nvs_set_u8(nvs, valid_key, 0)` — mark slot INVALID before touching data
10. `nvs_commit(nvs)` — flush the invalid flag to flash NOW
11. `nvs_set_blob(nvs, slot_key, &contact, sizeof(contact_entry_t))` — write 120-byte struct
12. `nvs_set_u8(nvs, valid_key, 1)` — mark slot VALID after data is fully written
13. `nvs_commit(nvs)` — final flush

WARNING: *The intermediate commit at step 2 is critical. Without it, a power loss between steps 1 and 2 might not flush the invalid flag to flash. On the next boot, the valid flag could still read 1, pointing at stale or corrupt data from a previous write. The double-commit pattern ensures that any interrupted write leaves the slot marked invalid.*

The `contact_entry_t` struct (120 bytes):

```
typedef struct {  
    char    name[32];           // 32 bytes  
    char    phone[16];          // 16 bytes  
    char    title[64];          // 64 bytes  
    uint32_t saved_at_epoch;     // 4 bytes — POSIX timestamp of save  
    uint8_t version;            // 1 byte — struct version for migration  
    uint8_t _pad[3];            // 3 bytes — alignment padding  
} contact_entry_t;             // = 120 bytes total
```

9. BLE Power Budget

Understanding the power cost of the BLE implementation is important for battery life planning in the v2 wearable form factor.

9.1 BLE advertising power

Parameter	Value	Notes
Advertising interval	100 ms	BLE_ADV_INTERVAL_MS in app_config.h
Advertising event duration	~0.4 ms	3 channels × ~0.13 ms each
Duty cycle	~0.4%	0.4 ms active / 100 ms interval
Peak TX current (ESP32-C3)	~80 mA	During TX burst
Average advertising current	~0.32 mA	80 mA × 0.4% duty cycle
Deep sleep base current	~5 µA	ESP32-C3 not yet using deep sleep in v1
Active MCU current (160 MHz)	~40-80 mA	Typical during BLE + display operations

DECISION: 100 ms advertising interval was chosen as a balance between discovery latency and power. At 100 ms, the average time for a scanning peer to discover this device is ~200 ms (scanner window must overlap at least one advertising event). A 500 ms interval would reduce advertising power by 5× but increase discovery latency to ~1 s, making the tap interaction feel sluggish. Since advertising only runs during the tap exchange window (typically 2-10 seconds), the absolute energy cost is low.

9.2 GATT exchange power

Phase	Duration	Current	Energy
GAP connection setup	~200-500 ms	~15 mA average	~3-7 mJ
GATT read × 3	~100 ms each, 300 ms total	~15 mA average	~4.5 mJ
GATT write × 3	~80 ms each, 240 ms total	~15 mA average	~3.6 mJ
GAP disconnect	~50 ms	~10 mA average	~0.5 mJ
Total exchange	~600-1100 ms	—	~12-16 mJ

NOTE: These are estimated values based on ESP32-C3 datasheet figures and typical NimBLE connection parameters. Actual values depend on connection interval negotiated by NimBLE

(default 7.5-15 ms) and the RF environment. Measure with a current probe on a real device for accurate battery life calculations.

9.3 Display power

Condition	Current
SSD1306 all pixels on	~20 mA
SSD1306 typical watch face (~30% pixels on)	~6-8 mA
SSD1306 display off (sleep)	~10 μ A
I2C bus during flush (~1 ms per frame)	Negligible — included in MCU active

9.4 v2 battery sizing guidance

For a 200 mAh LiPo (typical wrist wearable):

Scenario	Estimated battery life
Display on continuously, no BLE	$200 \text{ mAh} \div \sim 50 \text{ mA} = \sim 4 \text{ hours}$
Display on 10% of time, no BLE	~25 hours
10 TapShare exchanges per day, display off otherwise	Weeks — exchange is < 2 s, dominated by standby current
v2 with deep sleep + wake on button	Battery life measured in weeks to months

DECISION: v1 does not implement deep sleep. This is intentional — deep sleep adds significant complexity (RTC wakeup GPIO config, peripheral reinitialisation on wake, NimBLE stack restart) and is not needed to validate the core BLE exchange protocol. Deep sleep is a v2 power optimisation item.

10. Hardware Test Procedure — Tests 01 through 09

Every subsystem was validated independently before integration. Each test is a complete standalone ESP-IDF project in the tests/ directory. This incremental approach means that when a test fails, the fault is localised to the subsystem under test — not to the interaction of multiple untested components.

The test progression was designed so each test adds exactly one new untested component. No test depends on a component that has not already been verified by a previous test.

Test 01 — SSD1306 Display

Field	Detail
New component under test	ssd1306_driver — I2C bus init, init sequence, framebuffer, flush
Dependencies	None (first test)
What it does	Runs 16 sub-tests: clear screen, draw pixels, draw text at multiple scales, draw rectangles (outline and fill), combined drawing, edge pixels, text at boundaries
Key test	Test 12: white text then white rectangle at same coordinates. Verifies draw-order awareness. A dot-grid comparison was added after a draw-order bug was discovered here.
Pass criteria	All 16 sub-tests complete with expected visual output. No I2C errors in log.
Lesson learned	Draw order in framebuffer rendering matters. White-on-white conflicts are visually silent — test designs must use contrasting elements or known-position pixel inspection.

Test 02 — Button Input

Field	Detail
New component under test	button_driver — GPIO config, ISR, debounce, queue output
Dependencies	None independent
What it does	Logs every button event: which button, press or long-press, timestamp. Tests all 4 buttons.
Key test	Rapid press (faster than debounce threshold) must produce exactly one event. Hold for 800 ms must produce exactly one LONG_PRESS event, not a PRESS followed by a LONG_PRESS.
Pass criteria	Clean single event per physical gesture. No spurious events during hold. Long-press fires once at the threshold, not repeatedly.

Test 03 — Buzzer

Field	Detail
New component under test	buzzer_driver — LEDC PWM configuration, tone generation, silence
What it does	Plays each defined sound in sequence: button feedback, mode switch, share start, scan pulse, success chime, error tone, cancel tone.
Pass criteria	Audible tones match expected frequency and duration. No distortion. buzzer_silence() stops output immediately.

Test 04 — RTC

Field	Detail
New component under test	rtc_driver — internal RTC read/write, wifi_sync NTP
What it does	Sets a known time, reads it back every second, prints delta. If test_config.h is present, attempts WiFi connection and NTP sync, then prints the synced time.
Key test	After NTP sync, drift must be $< \pm 2$ s. Manual set/get round-trip must be exact.
Pass criteria	RTC holds time between reads. NTP sync sets clock within ± 2 s of reference.

Test 05 — Integration (Display + Button + Buzzer + RTC)

Field	Detail
What it does	Runs a minimal clock UI: OLED shows current time updated every second, button presses trigger buzzer tones, RTC provides time.
Why this test exists	Verifies that all four drivers work correctly when running simultaneously on the same I2C bus and GPIO bank. Cross-driver interference (e.g. I2C bus contention, GPIO state corruption) surfaces here.
Pass criteria	All peripherals respond correctly with no cross-driver interference. Display updates without glitches. Button events arrive without delay. Buzzer does not affect display or RTC.

Test 06 — BLE Advertise

Field	Detail
New component under test	ble_driver — NimBLE init, advertising, GATT server registration
What it does	Starts NimBLE, registers the 3-characteristic GATT service, begins advertising. Use nRF Connect or similar BLE scanner on a phone to verify.

Field	Detail
Key test	nRF Connect must show: device name 'tapexchange', TapShare service UUID, all 3 characteristics readable with placeholder values. Device must reappear after disconnect.
Pass criteria	Advertisement visible. All 3 GATT characteristics readable. Reconnects after disconnect without restart.

Test 07 — BLE Scan

Field	Detail
New component under test	ble_driver scanning path — observer role, scan callback, gate filter logging
Requires	Two boards (or one board + phone running an advertisement)
What it does	Starts active scanning and logs every discovered device. Applies the 4-gate filter and logs the result of each gate for every TapShare advertisement found.
Key test	A second board running test 06 must be discovered and log: Gate 1 PASS (UUID match), Gate 2 PASS/FAIL depending on distance, Gate 3 result with timestamp delta, Gate 4 role decision. Non-TapShare devices must be silently ignored.
Pass criteria	Gate results print correctly for each peer. Non-TapShare devices do not appear in filtered output.

Test 08 — BLE Connect

Field	Detail
New component under test	GAP connection establishment, role decision, disconnect
Requires	Two boards
What it does	Both boards activate sharing simultaneously. Role decision runs. Central initiates connection. Connection parameters are logged. Boards disconnect cleanly.
Key test	Run 5 times. Role assignment must be consistent — the same board must always be Central (lower tap timestamp wins). On a single board, scan timeout must fire after 8 s and transition to FAILED then IDLE.
Pass criteria	Consistent role assignment across 5 runs. Clean connect and disconnect. Single-board timeout → FAILED → IDLE works correctly.

Test 09 — Full TapShare End-to-End

Field	Detail
New component under test	Full GATT exchange, NVS save, contact readback
Requires	Two boards, each with unique profile data in test_config.h
What it does	Both boards simultaneously long-press SELECT. Full flow: SHARING → CONNECTING → EXCHANGING → DONE. Each board reads back the saved contact from NVS and prints it.
Key test 1	Contact printed on each board must match the other board's test_config.h profile.
Key test 2	Power cycle both boards. Contacts must persist in NVS after reboot.
Key test 3	Perform 6 exchanges (TAPSHARE_MAX_CONTACTS = 5). On the 6th exchange, slot 0 must be overwritten (FIFO rotation). Log must confirm ts_next wrapped to 0.
Pass criteria	Both boards reach BLE_STATE_DONE. NVS readback matches exchanged data. Contacts survive power cycle. FIFO rotation works at 6th exchange.
This is the current stable state of the repo	All 9 tests pass.

11. Future Work — Out of Scope for v1

The following components were designed, planned, and partially specified in the architecture documents but are explicitly out of scope for the current v1 firmware. They are documented here so that future contributors understand the intended direction.

11.1 Full UI layer (planned for v1 next upload)

The UI flow below is designed and will be implemented as the next firmware upload:

Screen	Trigger	Description
Watch face	Boot / auto-return	HH:MM:SS + date. Long-press SELECT = enter sharing.
Sharing	SELECT long-press	'SHARING...' animation. 8 s scan timeout. Long-press SELECT = cancel.
Share result	Exchange complete	'SAVED: [name]' + full contact details for 4 s, then auto-return.
Contact book	MODE press	Scrollable list of saved contacts. SELECT = next. Long-press SELECT = detail.
Contact detail	Long-press from contact book	Full name, phone, title. SELECT = back to list.

11.2 Alarm service

Designed but not implemented in v1. A simple alarm with NVS persistence, hour/minute set via buttons, and buzzer alert. Deferred because the core BLE exchange functionality was the primary validation goal of v1.

11.3 Stopwatch service

Designed but not implemented in v1. State machine: STOPPED → RUNNING → PAUSED, with lap timing. Deferred for the same reason as alarm — not relevant to validating the BLE exchange protocol.

11.4 v2 hardware upgrades

Upgrade	What changes	What stays the same
LIS3DH accelerometer tap trigger	Add lis3dh_driver.c. Replace button SELECT long-press in input_task with LIS3DH INT1 tap interrupt. ble_service API is unchanged.	All other drivers, all services, app layer

Upgrade	What changes	What stays the same
DS3231 external RTC	Replace rtc_driver.c only. API surface identical (rtc_get_time, rtc_set_hms).	All callers of rtc_driver unchanged
GC9A01 round colour display	Replace ssd1306_driver.c. Add LVGL (freed from RAM constraint with round display replacing OLED).	All non-display code
Companion phone app	ble_service_set_my_profile() is already implemented. App writes profile via GATT. Zero firmware changes needed.	Entire firmware unchanged
BLE time sync	rtc_driver API is time-source agnostic. BLE sync path calls rtc_set_hms() same as NTP path.	rtc_driver.c unchanged

12. Engineering Rules

These rules are enforced by code review. They are not optional style preferences — each one exists because a specific class of bug or complexity is prevented by following it.

Rule	Why it exists
One file does one thing. A driver file contains only driver code.	Prevents the 'helper function creep' that turns drivers into mixed-concern files that are impossible to unit test.
App calls service. Service calls driver. Driver touches hardware. Never skip a layer.	Layer skipping creates hidden dependencies. When a component is replaced, skipped layers must be found and fixed — often discovered at the worst time.
Tasks never call each other directly. Use a queue or event group.	Direct task calls from one task context to another are data races waiting to happen. Queues are the synchronisation primitive.
ISR functions are tiny. Send to queue and return. Zero logic in ISRs.	ISRs run with a tiny stack and restricted API. Any complexity in an ISR is a stability risk.
Every while(1) must contain a blocking call or vTaskDelay.	A while(1) without yield starves all equal and lower priority tasks. The watchdog eventually resets the chip.
All constants in app_config.h. All event types in app_events.h.	Scattered magic numbers are a maintenance hazard. Centralised constants mean hardware changes touch one file.
ESP_ERROR_CHECK every hardware call during bring-up.	Silent failures during bring-up produce confusing symptoms later. Asserting early gives a precise fault location.
BLE: check all 4 gates before ble_gap_connect().	A connection slot is a scarce resource. Never waste one on a peer that will fail the exchange.
BLE: read all characteristics first, then write.	The read-first contract prevents asymmetric half-exchanges. If any read fails, no writes occur.
NVS: write valid_flag=0 and commit before data. valid_flag=1 and commit after.	The double-commit pattern is the only way to detect and recover from power loss during an NVS write.
Replace placeholder UUIDs before hardware testing with other devices.	Generic UUIDs collide with other BLE devices. A collision causes gate 1 false-positives — random devices trigger TapShare connections.
Check stack high watermarks before any production build.	Stack overflow is the most common cause of silent task corruption. Minimum free stack must be > 200 bytes.

Appendix A — sdkconfig.defaults

The following settings are committed in `sdkconfig.defaults` and applied automatically on first build. Do not commit the generated `sdkconfig` file — it is gitignored.

Setting	Value	Purpose
CONFIG_BT_NIMBLE_ROLE_CENTRAL	y	Initiates GAP connections (Central role)
CONFIG_BT_NIMBLE_ROLE_PERIPHERAL	y	Accepts GAP connections, serves GATT (Peripheral role)
CONFIG_BT_NIMBLE_ROLE_OBSERVER	y	Active scanning
CONFIG_BT_NIMBLE_ROLE_BROADCASTER	y	Advertising
CONFIG_BT_NIMBLE_MAX_CONNECTIONS	1	TapShare is always a 1:1 exchange
CONFIG_BT_NIMBLE_NVS_PERSIST	n	No BLE bonding — saves NVS space for contacts
CONFIG_FREERTOS_HZ	1000	1 ms tick resolution — needed for accurate debounce timing
CONFIG_FREERTOS_ENABLE_BACKWARD_COMPATIBILITY	n	Forces use of correct ESP-IDF 5.x FreeRTOS API
CONFIG_ESP_TASK_WDT_ENABLED	y	Watchdog enabled — catches spinning tasks
CONFIG_FREERTOS_CHECK_STACKOVERFLOW	CANARY	Stack overflow detection with canary bytes

Appendix B — NVS Memory Budget

Item	Size	Notes
Default NVS partition	24 KB	Default ESP-IDF NVS partition
NimBLE internal keys	0 B	CONFIG_BT_NIMBLE_NVS_PERSIST=n — no bonding keys stored
WiFi credentials (SSID/password)	~0.5 KB	NVS write by esp_wifi on first connect

Item	Size	Notes
Own profile (3 STR keys)	~160 B	name + phone + title + NVS key overhead
Contact metadata (2 U8 keys)	~40 B	ts_count + ts_next FIFO pointer
5 contacts (slot blob + valid flag)	~750 B	5 × (120 B blob + 1 B flag + ~30 B NVS overhead)
Total used	~1.5 KB	6% of NVS partition
Available for expansion	~22.5 KB	Supports ~146 contacts before NVS partition is full

NOTE: `TAPSHARE_MAX_CONTACTS` is defined in `app_config.h`. Increasing it from 5 to 20 costs nothing in flash size — NVS overhead per slot is minimal. After changing the constant, run `idf.py erase-flash` to reset the NVS key layout.

End of Document — BLE Tap Exchange Firmware Architecture Rev 1.0

MIT License — Open Source